PRÁCTICA 1B - REDUNDANCIA Y ALMACENAMIENTO EN PROXMOX

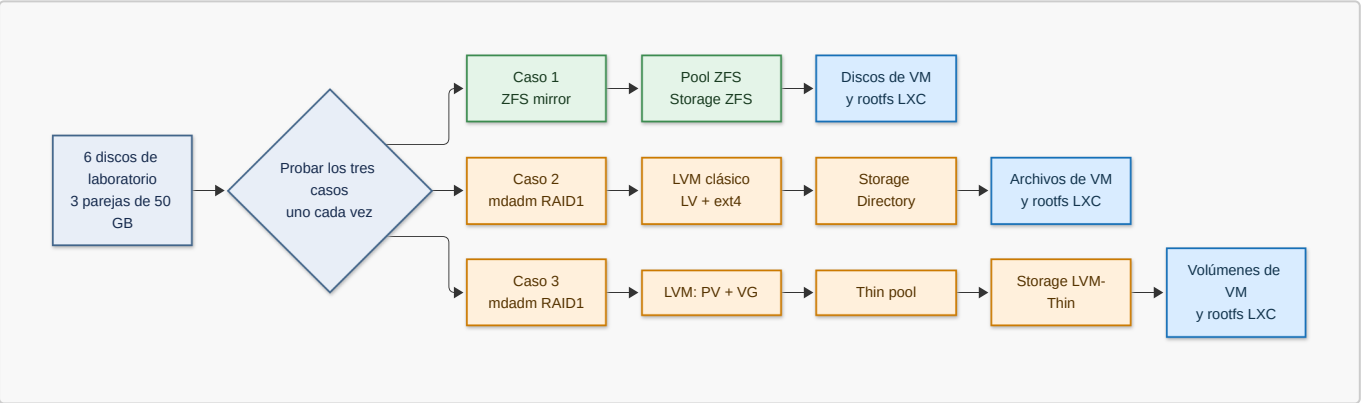
1. Introducción

Esta es la segunda de las cinco subprácticas de la práctica 1:

1. **Práctica 1a:** instalación de Proxmox VE.
2. **Práctica 1b:** redundancia y gestión del almacenamiento.
3. **Práctica 1c:** comparación entre VM y LXC.
4. **Práctica 1d:** despliegue de PaiportArbolado en VM y LXC.
5. **Práctica 1e:** gestión de cuotas y límites de capacidad en ZFS y LVM.

En la práctica anterior instalamos Proxmox VE. Ahora configuraremos almacenamiento redundante y seguiremos las capas entre los discos físicos y las VM o los contenedores.

Mapa visual de la práctica 1b



Los tres casos se realizan en la misma VM.

2. Objetivos

Configura y compara estas tres soluciones:

Caso 1: ZFS mirror

Discos físicos → ZFS mirror → pool ZFS → almacenamiento ZFS en Proxmox

Caso 2: LVM clásico sobre `mdadm` RAID1

Discos físicos → `mdadm` RAID1 → LVM clásico → LV con ext4 → almacenamiento Directory en Proxmox

Caso 3: LVM-Thin sobre `mdadm` RAID1

Discos físicos → `mdadm` RAID1 → LVM (PV y VG) → thin pool → almacenamiento LVM-Thin en Proxmox

En los casos 2 y 3, la redundancia la proporciona `mdadm`; **LVM clásico y LVM-Thin gestionan el espacio sobre el array, pero no añaden redundancia por sí mismos.**

Característica	ZFS mirror	`mdadm` RAID1 + LVM clásico	`mdadm` RAID1 + LVM-Thin
Redundancia	Nativa del pool	La proporciona `mdadm`	La proporciona `mdadm`
Aprovisionamiento	Según el formato del almacenamiento	Volúmenes convencionales y ext4	Discos virtuales thin-provisioned
Snapshots	Compatibles en ZFS	Dependen del formato de disco y del backend Directory	Compatibles con el backend LVM-Thin
Checksums de datos	Sí	No los proporciona esta combinación	No los proporciona esta combinación
Gestión	Integrada en ZFS	Capas separadas (`mdadm`, LVM y ext4)	Capas separadas (`mdadm` y LVM-Thin)

3. Conceptos clave

Término	Significado
Disco / partición	Dispositivo físico o división lógica que sirve de base al almacenamiento.
RAID1 / mirror	Mantiene una copia en cada miembro; con dos discos tolera la caída de uno.
PV, VG y LV	En LVM, el dispositivo físico, el grupo que lo reúne y el volumen lógico creado dentro del grupo. LVM no aporta redundancia.
LVM-Thin	Aprovisiona volúmenes según el espacio escrito, pero no aumenta la capacidad física. Vigila datos y metadatos: si el pool se llena, pueden fallar escrituras.
Dataset ZFS	Sistema de archivos dentro de un pool, montado según `mountpoint`. Para discos de VM y contenedores basta con registrar el pool en Proxmox.
Sistema de archivos	Organiza archivos sobre un volumen; por ejemplo, ext4 o ZFS.
ZFS	Sistema de archivos y gestor de almacenamiento con mirror, checksums, snapshots y resilver.

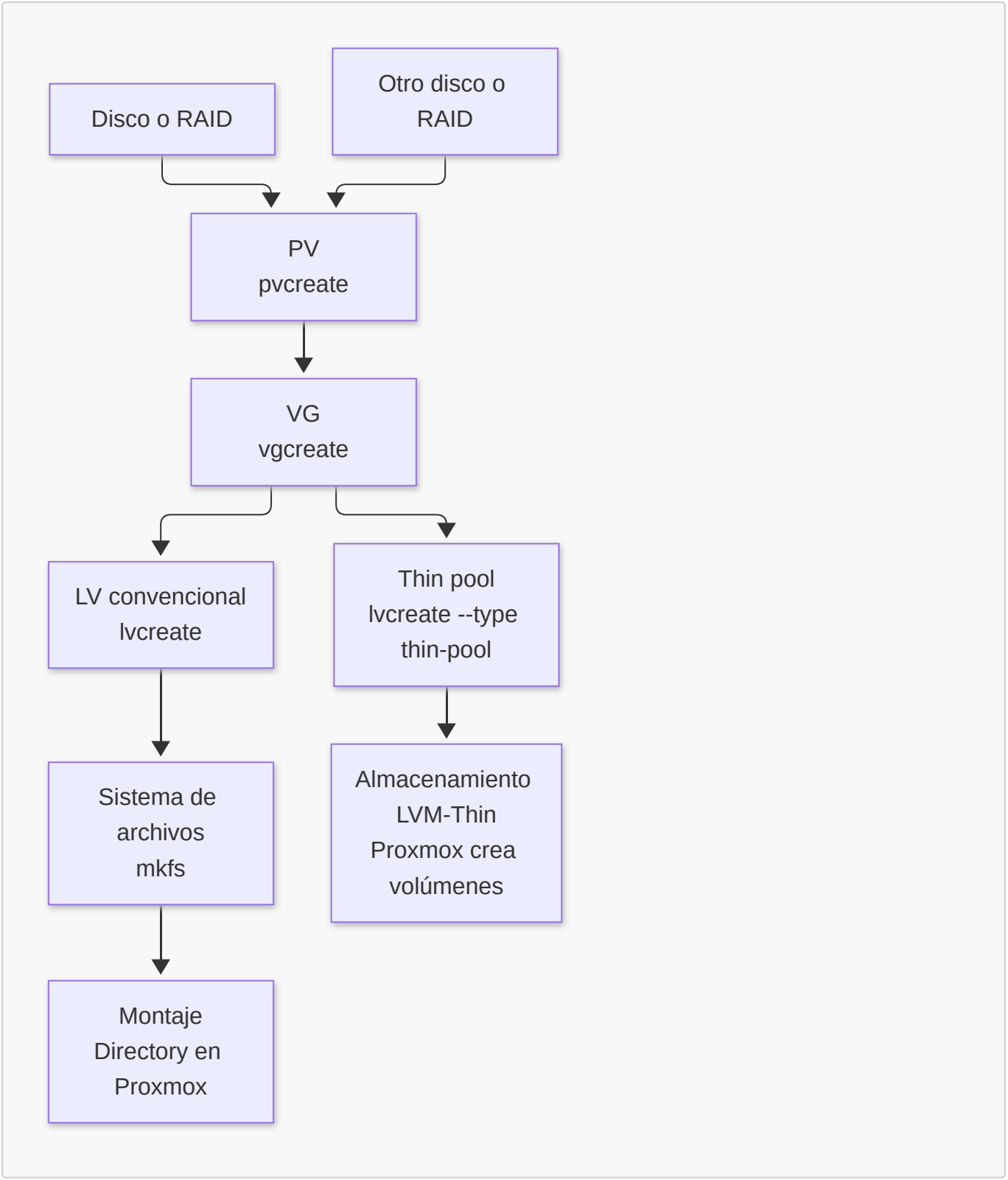
Guía rápida de LVM

Comandos y función:

Comando	Función
`pvcreate <dispositivo>`	Inicializa un disco, partición o dispositivo RAID para que LVM lo use como PV.
`vgcreate <VG> <PV>`	Agrupa uno o más PV en un grupo de volúmenes.

Comando	Función
`lvcreate ... <VG>`	Crea un volumen lógico convencional dentro del VG.
`mkfs.ext4 <LV>`	Crea un sistema de archivos sobre un LV convencional para poder montarlo.
`lvcreate --type thin-pool ... <VG>`	Crea un thin pool que puede alojar volúmenes thin sin formatear el pool con ext4.

En los casos 2 y 3, `mdadm` crea el RAID y LVM lo usa como PV. El VG del caso 2 contiene un LV con ext4; el del caso 3, un thin pool que Proxmox gestiona como almacenamiento de bloques.



El LV convencional se formatea y monta como directorio; Proxmox gestiona el thin pool como bloques. **No formatees ni montes el thin pool.** El espacio libre del VG permite ampliar LVs, pero ni LVM ni esos límites sustituyen a la redundancia o a las copias de seguridad.

4. Requisitos y preparación del laboratorio

Conserva el disco de sistema de la VM y añade seis discos de laboratorio: tres parejas de 50 GB, una para cada caso. **No reutilices ni modifiques el disco de sistema.**

Requisitos: Proxmox VE instalado en VirtualBox y acceso como `root` a la Shell del nodo. Usa VDI dinámicos de 50 GB (o fijos, que también son válidos). RAID1 ofrece aproximadamente la capacidad de un disco, no la suma. Con VDI dinámicos, el anfitrión debe conservar espacio libre para evitar errores de escritura.

4.1. Crear e identificar los discos en VirtualBox

Con la VM apagada, abre **Configuración** → **Almacenamiento**, selecciona el controlador SATA y usa **Añadir disco duro** → **Crear**. Para cada disco:

1. Crea VDI de 50 GB, preferiblemente **Reservado dinámicamente** (el tamaño fijo también sirve).
2. Nómbralos según la pareja:

Caso	Nombres de archivo VDI
1. ZFS mirror	`ZFS_raid1_1.vdi`, `ZFS_raid1_2.vdi`
2. `mdadm` RAID1 + LVM clásico	`LVM_raid1_1.vdi`, `LVM_raid1_2.vdi`
3. `mdadm` RAID1 + LVM-Thin	`LVM_thin_raid1_1.vdi`, `LVM_thin_raid1_2.vdi`

Antes de crear o formatear almacenamiento, confirma en VirtualBox y con `lsblk` cuáles son los discos:

```
lsblk
```

Los nombres `/dev/sdb` y `/dev/sdc` son ejemplos y pueden cambiar. **No continúes si no identificas con certeza los discos:** crear o formatear destruye sus datos.

Si los discos de laboratorio tienen firmas anteriores, límpialas solo en esos dispositivos:

```
wipefs -a /dev/sdb
wipefs -a /dev/sdc
```

Solo haz esto sobre los dispositivos de laboratorio, nunca sobre el disco del sistema de Proxmox.

5. Parte práctica: configurar los tres casos

Realiza un caso cada vez: conecta solo su pareja de discos y apaga la VM de Proxmox antes de cambiarla.

5.1. ZFS con mirror

1. Identifica los dos discos de laboratorio:

```
lsblk
```

2. Crea el pool con los dispositivos confirmados. No continúes si no estás seguro:

```
zpool create -o ashift=12 zfs-raid1-storage mirror /dev/sdb /dev/sdc
```

Parámetro	Función
`ashift=12`	Alinea la asignación a sectores de 4 KiB.
`zfs-raid1-storage`	Nombre del pool.
`mirror`	Espejo formado por los dos discos indicados.

3. Comprueba el estado y la capacidad:

```
zpool status zfs-raid1-storage
zpool list zfs-raid1-storage
```

También puedes crearlo desde **nodo → Disks → ZFS → Create: ZFS**, seleccionando los discos y **Mirror**. Usa CLI o GUI, no ambos; verifica con `zpool status`.

El pool ya está disponible al crearlo. Para discos de VM o contenedores, regístralo en Proxmox; no hace falta crear ni montar un dataset manualmente.

4. Da de alta el pool desde la interfaz web: **Datacenter → Storage → Add → ZFS**. Configura:

- **ID:** `zfs-lab`
- **ZFS Pool:** `zfs-raid1-storage`
- **Content:** `Disk image` y, si se crearán contenedores, `Container` (rootdir)
- **Node:** el nodo actual
- Activa **Thin provision** si aparece en el formulario.

Proxmox gestiona las estructuras de VM y contenedores. No crees datasets `images`/`backup` ni uses este almacenamiento como destino de copias de seguridad.

5. Comprueba el registro y el aprovisionamiento disperso:

```
cat /etc/pve/storage.cfg
```

En la entrada `zfspool: zfs-lab` debe figurar `sparse 1`. Si la interfaz no ofrece Thin provision, verifica o ajusta la entrada existente para que incluya esa opción. Conserva las demás opciones generadas, no dupliques el almacenamiento y usa espacios para indentar:

```
zfspool: zfs-lab
        pool zfs-raid1-storage
        content images,rootdir
        sparse 1
```

`sparse 1` evita reservar de antemano todo el tamaño virtual. Confirma también el almacenamiento en **Datacenter → Storage** y consulta `zpool status`.

5.2. Caso 2: LVM clásico sobre `mdadm` RAID1

`mdadm` aporta el RAID1; LVM crea un LV, que se formatea con ext4 y se monta. Proxmox registra la ruta montada como **Directory** (admite `images` y `rootdir`). El backend **LVM** gestiona LVs directamente y admite `images`; para volúmenes thin se usa **LVM-Thin**.

1. Identifica los discos visibles en el nodo:

```
lsblk
```

2. Verifica que los discos que vas a usar son los de laboratorio y no el disco del sistema. En el ejemplo se usan `/dev/sdb` y `/dev/sdc`, pero puede que sean otros. Si no estás seguro, no continúes.

3. Crea el array RAID1:

```
mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc
```

Si los discos tenían datos o firmas previas, límpialos antes con `wipefs -a` sobre cada dispositivo de laboratorio; en caso contrario, el array podría llegar a crearse sobre datos existentes y dar resultados engañosos.

4. Comprueba la estructura creada:

```
mdadm --detail /dev/md0
cat /proc/mdstat
```

En este punto se debe ver que `/dev/md0` está activo y sincronizando los datos entre ambos miembros del array.

Espera a que termine la sincronización inicial antes de continuar. Registra el array para que pueda volver a ensamblarse después de reiniciar Proxmox. Comprueba primero si ya existe una entrada equivalente en `/etc/mdadm/mdadm.conf`; añade la salida de `mdadm --detail --scan` solo si no está registrada y actualiza el initramfs:

```
mdadm --detail --scan
nano /etc/mdadm/mdadm.conf
update-initramfs -u
```

En el editor, copia la línea `ARRAY` que muestra el primer comando y guárdala en `mdadm.conf` solo si no hay ya una entrada con el mismo UUID. No reemplaces el contenido existente del archivo.

5. Convierte el array en un volumen físico de LVM:

```
pvcreate /dev/md0
pvs
```

6. Crea un grupo de volúmenes y un volumen lógico de 32 GiB para el sistema de archivos de Proxmox:

```
vgcreate raid1-vg /dev/md0
vgs

lvcreate -L 30G -n proxmox-data raid1-vg
lvs
```

El tamaño útil del RAID1 será aproximadamente el de un disco (menos metadatos). El LV empieza con 30 GiB y se ampliará a 32 GiB en el paso 8; así queda espacio en el VG para el LV independiente de 1 GiB creado en la práctica 1e. La VM y el contenedor de las prácticas 1c y 1d usarán discos raíz de 12 GiB cada uno; no amplíes estos tamaños sin volver a comprobar el espacio libre.

7. Formatea el LV de datos y móntalo como sistema de archivos:

```
mkfs.ext4 /dev/raid1-vg/proxmox-data
mkdir -p /mnt/raid1-lab
mount /dev/raid1-vg/proxmox-data /mnt/raid1-lab
df -h /mnt/raid1-lab
```

Haz que el montaje persista después de reiniciar. Obtén el UUID del sistema de archivos y añade la entrada indicada a `/etc/fstab`:

```
blkid /dev/raid1-vg/proxmox-data
```

Usa el UUID que devuelve el comando en esta línea (sustituye `<UUID>`):

```
UUID=<UUID> /mnt/raid1-lab ext4 defaults,nofail,x-systemd.device-timeout=5s
0 2
```

`nofail` permite arrancar Proxmox cuando la pareja de discos LVM no está conectada para probar ZFS; `is_mountpoint 1` hará que el almacenamiento Directory figure offline en ese caso. Comprueba el montaje:

```
mount -a
```

```
df -h /mnt/raid1-lab
```

8. Amplía el LV en 2 GiB y extiende el sistema de archivos ext4:

```
lvextend -L +2G /dev/raid1-vg/proxmox-data
resize2fs /dev/raid1-vg/proxmox-data
df -h /mnt/raid1-lab
```

9. Registra el punto de montaje desde **Datacenter → Storage → Add → Directory** (no el tipo **LVM**):

- **ID:** `lvm-raid1-dir`
- **Directory:** `/mnt/raid1-lab`
- **Content:** `Disk image` y `Container` (rootdir)
- **Node:** el nodo actual
- Activa **Is mountpoint** (o configura `is_mountpoint 1`) para que Proxmox no use accidentalmente el directorio vacío del disco de sistema si el LV no está montado.

Comprueba que `/etc/pve/storage.cfg` contiene una entrada similar a:

```
dir: lvm-raid1-dir
        path /mnt/raid1-lab
        content images,rootdir
        is_mountpoint 1
```

```
Caso 2: discos → mdadm RAID1 → PV/VG → LV → ext4 → ruta Directory
Caso 3: discos → mdadm RAID1 → PV/VG → thin pool → LVM-Thin
```

En el caso 2, Proxmox guarda `images` y `rootdir` en `/mnt/raid1-lab`; comparten el espacio de ext4. En el caso 3, Proxmox crea volúmenes de bloque separados en el thin pool. `mdadm` aporta la redundancia; LVM no. Thin provisioning no aumenta la capacidad física: vigila `Data%` y `Meta%`, pues si se llena el pool pueden fallar las escrituras.

5.3. Caso 3: LVM-Thin sobre `mdadm` RAID1

Este caso utiliza los dos VDI nuevos `LVM_thin_raid1_1.vdi` y `LVM_thin_raid1_2.vdi`. La redundancia procede de `mdadm`; el thin pool proporciona volúmenes virtuales thin para VM y contenedores. No se crea un LV ext4 ni un almacenamiento Directory.

1. Apaga Proxmox y conecta únicamente esta pareja. Arranca la VM e identifica los dispositivos de laboratorio con `lsblk`; los nombres `/dev/sdb` y `/dev/sdc` son solo ejemplos.
2. Comprueba cuidadosamente que son los dos VDI nuevos, sin datos que necesites. Si tienen firmas anteriores, límpialas solo en esos discos:

```
wipefs -a /dev/sdb
wipefs -a /dev/sdc
```

3. Crea el array RAID1 dedicado a este caso:

```
mdadm --create /dev/md1 --level=1 --raid-devices=2 /dev/sdb /dev/sdc
watch cat /proc/mdstat
```

Espera a que termine la sincronización inicial y sal de `watch` con `Ctrl+C`. Registra el array para que pueda volver a ensamblarse después de reiniciar Proxmox. Comprueba primero si ya existe una entrada equivalente en `/etc/mdadm/mdadm.conf`; añade la salida de `mdadm --detail --scan` solo si no está registrada y actualiza el initramfs:

```
mdadm --detail --scan
nano /etc/mdadm/mdadm.conf
update-initramfs -u
```

4. Inicializa el array como PV y crea un VG distinto del utilizado en el caso 2:

```
pvcreate /dev/md1
vgcreate lvmthin-raid1-vg /dev/md1
pvs
vgs
```

5. Crea un thin pool que deje espacio libre en el VG para metadatos y crecimiento. Con discos de 50 GB, reservar el 90 % del VG es una referencia inicial, no un objetivo de capacidad que debas forzar si `vgs` muestra menos espacio:

```
lvcreate --type thin-pool -l 90%VG -n data lvmthin-raid1-vg
lvs -a -o+segtype,lv_size,data_percent,metadata_percent
```

El tamaño útil del RAID1 será aproximadamente el de un disco (50 GB decimales, unos 46,6 GiB antes de metadatos). El tamaño del thin pool será menor porque se deja margen al VG y se necesita espacio para metadatos. No crees discos thin que sobrepasen sin control el espacio físico disponible y vigila `Data%` y `Meta%`: un pool lleno puede provocar errores de escritura en sus volúmenes.

6. Registra el thin pool en **Datacenter → Storage → Add → LVM-Thin**:

- **ID:** `lvmthin-raid1`
- **Volume group:** `lvmthin-raid1-vg`
- **Thin pool:** `data`

- **Content:** `Disk image` y `Container` (rootdir), si se usarán ambos
- **Node:** el nodo actual

No elijas el tipo **LVM** convencional ni **Directory**. Comprueba que `/etc/pve/storage.cfg` contiene una entrada equivalente:

```
lvmthin: lvmthin-raid1
        thinpool data
        vgname lvmthin-raid1-vg
        content images,rootdir
```

7. Verifica el array, el VG, el thin pool y el almacenamiento registrado:

```
mdadm --detail /dev/md1
cat /proc/mdstat
pvs
vgs
lvs -a -o+segtype,lv_size,data_percent,metadata_percent
pvesm status
```

Comprueba también **Datacenter** → **Storage** y que el estado del almacenamiento `lvmthin-raid1` sea activo.

5.4. Verificación final

Después de crear cada caso, comprueba que la capa de almacenamiento queda visible en Proxmox y que sus herramientas de diagnóstico muestran el estado correcto:

```
pvs
vgs
lvs
lsblk
```

Si has elegido ZFS, usa además:

```
zpool status zfs-raid1-storage
zpool list zfs-raid1-storage
```

Si has configurado el caso 2, usa:

```
mdadm --detail /dev/md0
cat /proc/mdstat
```

Si has configurado el caso 3, usa:

```
mdadm --detail /dev/md1
cat /proc/mdstat
lvs -a -o+segtype,lv_size,data_percent,metadata_percent
pvesm status
```

6. Simulación de fallo de disco

Prueba cada caso con sus propios discos y tras la sincronización inicial. Comprueba siempre el array y el miembro exactos antes de actuar.

6.1. Casos 2 y 3: RAID1 con `mdadm`

Caso 2: `/dev/md0`. Caso 3: `/dev/md1`. Sustituye `<ARRAY>` y `<DISCO>` por los valores que confirme `mdadm --detail` (no asumas que el disco es `/dev/sdb`). Marca y retira un miembro:

```
mdadm --manage <ARRAY> --fail <DISCO>
mdadm --manage <ARRAY> --remove <DISCO>
```

Verifica que sigue activo en estado degradado:

```
cat /proc/mdstat
mdadm --detail <ARRAY>
```

Si el miembro sigue conectado, reincorpóralo y espera a que termine la reconstrucción:

```
mdadm --manage <ARRAY> --add <DISCO>
cat /proc/mdstat
```

6.2. ZFS mirror

Usa la ruta exacta que muestra `zpool status -P`:

```
zpool status -P zfs-raid1-storage
```

Sustituye `<DISCO>` por la ruta exacta del miembro que quieres probar. Desconéctalo lógicamente y comprueba que el pool sigue disponible, aunque degradado:

```
zpool offline zfs-raid1-storage <DISCO>
```

```
zpool status -P zfs-raid1-storage
```

Reincorpora el miembro:

```
zpool online zfs-raid1-storage <DISCO>
zpool status -P zfs-raid1-storage
```

`offline` simula una retirada lógica, no una avería física. Comprueba el estado hasta que termine el resilver.

7. Prueba de continuidad del servicio

Repite la prueba en cada almacenamiento:

1. Crea y arranca un LXC con rootfs en el almacenamiento del caso.
2. Comprueba que está activo y escribe un registro:

```
pct status <CTID>
pct exec <CTID> -- sh -c 'date >> /root/continuidad.log'
```

3. Simula la caída de un miembro siguiendo la sección 6. Mientras el almacenamiento está degradado, repite la escritura y comprueba el archivo:

```
pct exec <CTID> -- sh -c 'date >> /root/continuidad.log'
pct exec <CTID> -- cat /root/continuidad.log
```

4. Restaura el miembro, espera a que termine la reconstrucción o resilver y confirma que el LXC sigue activo.

8. Comparación de resultados

Resume los datos observados en cada caso:

Caso	Redundancia	Capacidad útil	Estado degradado y recuperación	Funciones destacadas
ZFS mirror	ZFS			Checksums, snapshots y resilver
`mdadm` + LVM clásico	`mdadm`			LV convencional y ext4 en Directory
`mdadm` + LVM-Thin	`mdadm`			Volúmenes thin; anota también `Data%` y `Meta%`

Thin provisioning no añade redundancia ni capacidad física. Justifica qué alternativa elegirías para el escenario.

9. Conclusión técnica

ZFS mirror y `mdadm` RAID1 mantienen el servicio ante la caída de un miembro, pero no sustituyen a las copias de seguridad: fallos del sistema, corrupción o errores humanos aún pueden exigir una restauración. Elige según la gestión, recuperación, snapshots y compatibilidad que necesite el escenario.

10. Reflexión y preguntas

Responde con tus propias palabras:

1. ¿Qué diferencia existe entre un volumen lógico y un sistema de archivos?
2. ¿Qué ventajas tiene RAID 1 frente a RAID 0?
3. ¿Por qué la redundancia no sustituye a las copias de seguridad?
4. ¿Qué función tiene un `zpool` en ZFS?
5. ¿Qué diferencia existe entre un dataset y un zvol?
6. ¿Qué es un mirror y qué fallo puede tolerar el mirror de dos discos de esta práctica?
7. ¿Qué diferencia hay entre LVM-Thin y ZFS a la hora de almacenar discos de VM?
8. ¿Por qué LVM no aporta redundancia por sí solo?
9. ¿Qué diferencia existe entre el RAID gestionado por `mdadm` y un mirror ZFS?
10. ¿Qué información aporta el estado del array o del pool después de simular un fallo?
11. ¿Qué representa el disco virtual que ve la VM y qué capas puede crear dentro de él?
12. ¿Qué ventajas y riesgos tiene el aprovisionamiento thin? ¿Qué indican `Data%` y `Meta%`?
13. ¿Qué capas comparten los casos 2 y 3, y en qué se diferencian a partir del VG?

11. Entrega

La entrega debe incluir, como mínimo:

- Capturas de la configuración de los discos de laboratorio en VirtualBox.
- Captura de `lsblk` donde se identifiquen los discos añadidos.
- Capturas del estado y la capacidad de los tres casos: `zpool status` y `zpool list`; `mdadm --detail`, `cat /proc/mdstat`, `pvs`, `vgs` y `lvs`; y, para LVM-Thin, `lvs -a -o+segtype,lv_size,data_percent,metadata_percent`.
- Captura de **Datacenter** → **Storage** mostrando cada uno de los tres almacenamientos registrados.
- Explicación de la capacidad útil y las capas de cada caso.
- Evidencia y explicación de las tres simulaciones de fallo y de la recuperación de cada miembro.
- Respuestas redactadas a las preguntas de reflexión.
- Conclusión sobre qué tecnología resulta más útil para el escenario estudiado y por qué.